

There are **100** points possible.

Honor Pledge: I affirm that I will not give or receive any unauthorized help on this exam, and that all work will be my own.

Indicate agreement by typing or signing your name: Josiah Schmitz

1. (14 pts) Let G be the grammar defined by the following rules (the start symbol is S):

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$aB \rightarrow b$

- a) Is the string $aaaab$ a member of the language defined by G ? If so, show a derivation. If not, briefly explain why not.

Answer: Yes. Derivation is as follows:

$S \rightarrow AB$	$A \rightarrow aA$
$\rightarrow aAB$	$A \rightarrow aA$
$\rightarrow aaAB$	$A \rightarrow aA$
$\rightarrow aaaAB$	$A \rightarrow aA$
$\rightarrow aaaaAB$	$A \rightarrow a$
$\rightarrow aaaaaB$	$aB \rightarrow b$
$\rightarrow aaaaab$	

- b) Is this grammar ambiguous? *Justify your answer.*

Answer: No, it is unambiguous. For any given string produced by this grammar, there is only one parse tree that leads to that string.

2. (6 pts) In mathematical expressions, parentheses can be arbitrarily added to sub-expressions. For example, all of the following expressions are equal:

$(7 + 5)$ $((7 + 5))$ $((7) + (5))$ $7 + 5$

In Racket, however, the following two expressions are *not* equal:

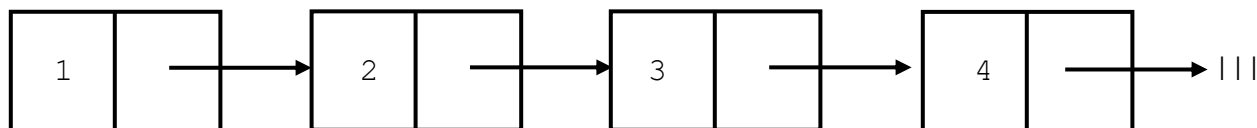
$(+ 7 5)$ $((+ 7 5))$

Which expression (the first or the second) would result in an error **and why**?

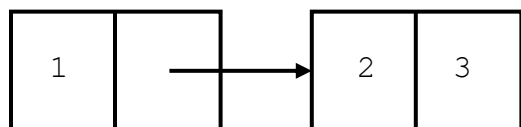
Answer: The second will result in an error. This is because Racket will treat the first one as a simple procedure that returns the result of $7 + 5$ (12). When the additional parentheses are added to the second expression, Racket expects an additional procedure that will act on the result of the first expression, $(7 + 5)$. Because no procedure has been added that will apply said result (like “symbol?” or “cons”), Racket returns an error.

3. (20 pts) For each of the following Racket forms, show the resulting list structure in **boxes-and-arrows** representation.

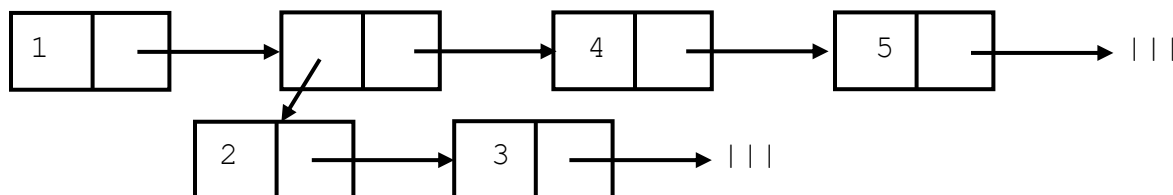
a) `(cons 1 (cons 2 (cons 3 (cons 4 '()) )))`



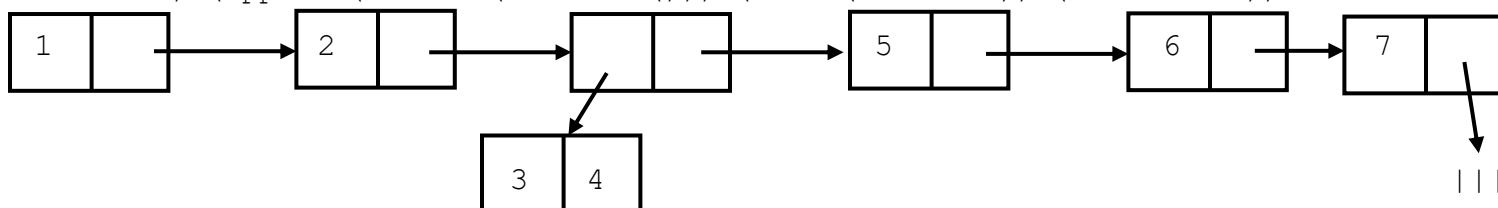
b) `(cons 1 (cons 2 3))`



c) `(cons 1 (cons (cons 2 (cons 3 '())) (cons 4 (cons 5 '()))))`



d) `(append (cons 1 (cons 2 '())) (list (cons 3 4)) (list 5 6 7))`



4. (6 pts) Given the following Racket form:

```
(+ (* (/ (string->number "7") 2) (- bar foo)) 9)
```

- a) If this form were evaluated by Racket, what token would be evaluated first? You may assume that arguments are evaluated in **left-to-right** order. (Tokens include +, *, /, string->number, "7", 2, -, bar, foo, and 9)

Answer: 'string->number' would be evaluated first since it is the furthest left operand, and operands are evaluated before operators.

- b) Which token would be evaluated next (second)?

Answer: "7" would be evaluated next since it is just right of string->number.

- c) Which token would be evaluated third?

Answer: 2 would be evaluated next by the same logic as above.

5. (10 pts) Write the Racket function `div-by` that accepts a single numeric argument and returns a list of all numbers less than 15 that the argument is divisible by.

(add to Racket file) your code for the function `div-by`

6. (20 pts) Consider the problem `count-evens`, defined as follows: Input: a list of integers. Output: the number of even integers in the list. For example, `(count-evens '(1 4 3 7 8))` should return the value 2 because the list contains two even numbers.

- a) Describe a recursive strategy for solving this problem in Racket *without using any helper functions or parameters other than the input list*. Be sure to describe how you would break the problem down into sub-problems, and how each sub-problem would be solved.

(add to Racket file) your code for function `count-evens`

Answer: I need to start by creating a base case and a recursive step to the function. For the base case, since the function takes a list as input, it should return only when the list is done being searched. This can be done by having the function return when the input list is empty. I also need the return value for the base case, and since the function should return the number of even values in the list, the base case should return 0. For the recursive step, I need two branches: one for if the next value is even and one for if it's not. I can check for evenness by using modular arithmetic. If the next value of the list (the 'car') is even, I'll make a recursive call to 'count-evens' using the rest of the list (the 'cdr') and add 1 to its output to keep track of the number of evens. Otherwise, if the next value is odd, I'll make the same recursive call without adding anything. This will allow for the number of evens in the list to accumulate if the next element using the return value of 'count-evens' itself.

- b) Now suppose that the input list can be nested. For example, you can call `(count-evens '(1 (2 3) (4 (5 6))))`. How would you modify your strategy from part-a to provide a correct algorithm for nested lists?

(add to Racket file) your modified code for `count-evens`

Answer: Very little of the code needs to be modified for a version with sub lists. I will simply need to add a check to my recursive step that sees if the next element of the list is a list or not. If it is a list, I can make a recursive call of 'count-evens' to the sub list. Then I can just add this return value to the return value of the recursive call to the rest of the list. This method should allow the function to finish running on a sub list entirely before trying to move on to the next element of the main list. It should also allow for sub lists of sub lists to be checked properly.

7. (8 pts) What is the difference between tail-call and non-tail-call recursion? Which one is more efficient in Racket, and why?

Answer: Tail-call recursion is when the very last operation before returning in a recursive function is the recursive call. Conversely, non-tail-call recursion involves the recursive call existing anywhere but as the final operation in the recursive function. Tail-call recursion is more efficient because Racket automatically recognizes when tail-call recursion is implemented and will internally convert the function to an iterative loop at runtime. And in non-tail-call recursion, Racket needs to store all the instructions after the recursive call so that it can apply them once the recursive call is done. This requires additional space from the system, making non-tail-call recursion less efficient.

8. (10 pts) Consider the following Racket function:

```
(define sumsquares
  (lambda (lst)
    (if (null? lst)
        0
        (+ (* (car lst) (car lst)) (sumsquares (cdr lst)))
    )))
```

Is this a tail-call? If so, explain why. If not, re-write the function so that it is a tail-call

(add to Racket file).

Answer: It is a tail call. This is because 'sumsquares' in the last expression, which is the recursive call, is the last operation called within the lambda expression defining the overall recursive function.

9. (6 pts) a) What are Racket structures. Write some of the features of the Racket structures.

Answer: A structure in racket is almost identical to objects like classes in other languages. A structure has data members which are the various fields that describe the Racket structure. By default, Racket automatically defines many of the features in a structure that would normally be manually created in other object-oriented focused languages, such as Java. These features include access functions, modifier functions (also known as getters and setters in other languages), and constructors for the structure itself.

b) Write a program that creates a structure "car" that has data members: type, model, color, num-doors, top-speed. Create three objects for three car makes, for example (ford, honda, ...). The data member type is a truck or sedan. Put values of your choice for the rest of the data members The program then accesses the last three data members of each object to make changes (add to Racket file).